

Per executar aquest servidor:

- 1) Guardem el codi en un fitxer anomenat **server.js**.
- 2) Obrim una terminal en el directori on hem desat l'script `server.js` i executem: `node server.js`.

Visitem <http://localhost:3000> al navegador per veure la resposta.

1.3 Sintaxi del llenguatge: variables, operadors, condicionals, bucles

Un cop entès el paper de JavaScript dins de l'arquitectura web i coneguts els seus orígens, és el moment de centrar-nos en la seva **sintaxi bàsica**. La sintaxi és el conjunt de regles que defineixen com s'ha d'escriure i estructurar el codi perquè el navegador (o qualsevol entorn d'execució) el pugui interpretar correctament. Aprendre aquests fonaments és imprescindible per començar a programar, ja que constitueixen les peces bàsiques amb què construirem instruccions més complexes i aplicacions completes.

1.3.1 Variables i tipus

JavaScript és un llenguatge **feblement tipat**, és a dir, no indiquem de quin tipus és cada variable quan la declarem. Totes les variables poden contenir qualsevol tipus de dada i poden ser reassignades. Això té els seus avantatges i inconvenients.

Les variables són espais de memòria on emmagatzemem dades de manera temporal, i hi podem accedir en qualsevol moment durant l'execució del nostre programa. Hi ha diversos tipus i classes, que veurem tot seguit.

Per definir una variable en JavaScript, utilitzem la paraula reservada *let* i li donem un nom:

```
1 let elMeuValor;
```

També podem assignar-li un valor directament:

```
1 let dada = 5;
```

Com mostrar dades a l'usuari?

Per mostrar dades a l'usuari, tenim diferents mecanismes. Els més bàsics són mostrar-los per consola (`console.log(dada)`) o si ens trobem en un document web els podem mostrar per pantalla (`document.write(dada)`) o fins i tot fer-los sortir en una finestra emergent del navegador (`alert(dada)`).

O fer-ho en una línia posterior:

```
1 let dada;  
2 dada = 5;
```

Al llarg de les lliçons, sovint veurem declarar variables amb la partícula *const*. Això serveix per declarar una variable amb un valor constant i no podrà ser canviat excepte que la variable sigui de tipus objecte o array. Per exemple:

```
1 const PI = 3.1416;
```

Finalment, també val la pena conèixer que antigament es declaraven les variables amb la partícula `var`. Malgrat que encara funciona es desaconsella el seu ús.

Els noms de les variables haurien de ser el més **descriptius** possible per entendre millor què contenen. Han de començar sempre per una lletra, `$` o `_`, però **mai per números** ni altres caràcters especials. JavaScript distingeix entre majúscules i minúscules, per tant, `meuValor` i `MeuValor` serien dues variables diferents.

JavaScript disposa de **set tipus primitius** per emmagatzemar dades:

- **number**: s'utilitza per a **valors numèrics**, tant enters com decimals. Exemples:

```
1 let numEnter = 5;  
2 let numDecimal = 3.14;  
3 console.log(typeof numDecimal); // "number"
```

- **boolean**: només pot contenir `true` o `false`, i s'utilitza per representar estats:

```
1 let on = true;  
2 let off = false;
```

- **string**: representa cadenes de caràcters, entre cometes simples o dobles:

```
1 let missatge = "Hola món";  
2 let altreMissatge = 'Bon dia';
```

- **undefined**: És el valor que pren una variable quan **no se li ha assignat cap valor**, o s'ha assignat explícitament `undefined`:

```
1 let noDefinida;  
2 let tambéIndefinida = undefined;
```

- **null**: valor assignat que representa **absència intencionada de valor**:

```
1 let resposta = null;  
2 console.log(resposta); // null
```

- **bigint**: per treballar amb enters molt grans:

```
1 let granNumero = 1234567890123456789012345678901234567890n;  
2 console.log(granNumero); // →  
   1234567890123456789012345678901234567890n  
3 console.log(typeof granNumero); // "bigint"
```

- **symbol**: identificadors únics, sovint per propietats d'objectes:

```
1 const id = Symbol("usuariID");  
2 console.log(id); // Symbol(usuariID)  
3 console.log(typeof id); // "symbol"
```

Altres tipus

A part dels tipus primitius, JavaScript també utilitza **objectes** per representar estructures de dades més complexes. Tot en JavaScript és un objecte (incloses les funcions). Alguns objectes comuns són:

- `Object`
- `Array`
- `Date`

Tots ells hereten de la classe `Object`.

1.3.2 Capturar dades

Els valors desats en les variables estudiades a la secció anterior poden haver estat introduïts pels desenvolupadors o bé provenir directament de l'usuari. Capturar informació de l'usuari és fonamental per fer programes interactius i adaptatius. Tot i que la forma més habitual en aplicacions web són els formularis (que tractarem més endavant), hi ha altres mètodes senzills i molt útils per obtenir dades, tant en entorns de navegador com en NodeJS.

Des del navegador

En JavaScript que s'executa al navegador podem interactuar amb l'usuari de manera immediata a través de finestres emergents:

- **`prompt('Pregunta')`**. Permet mostrar un quadre de diàleg amb una pregunta i un camp de text per introduir dades. Retorna sempre una **cadena** de text, encara que l'usuari escrigui un número.

```
1 let nom = prompt('Com et dius?');  
2 console.log('Hola, ' + nom + '!');
```

- **`confirm('Estàs segur?')`**. Mostra un quadre de diàleg amb els botons **OK** i **Cancel**, retornant un valor **boolean** (`true` si l'usuari prem OK, `false` si prem Cancel).

```
1 let confirmar = confirm('Vols continuar amb l'acció?');  
2 if (confirmar) {  
3   console.log('Acció confirmada.');4 } else {  
5   console.log('Acció cancel·lada.');6 }
```

- **`alert('Missatge')`**. Tot i que no captura dades, és útil per mostrar informació immediata a l'usuari. Sovint es combina amb `prompt` o `confirm`.

```
1 alert('Benvingut a la nostra pàgina!');
```

Des de la consola amb NodeJS

En entorns fora del navegador, com NodeJS, podem capturar dades de l'usuari a través de la consola. La manera més senzilla és mitjançant el mòdul `readline`:

```
1 const readline = require('readline');
2
3 const rl = readline.createInterface({
4   input: process.stdin,
5   output: process.stdout
6 });
7
8 rl.question('Com et dius? ', (nom) => {
9   console.log('Hola, ${nom}!');
10  rl.close();
11 });
```

- `readline.createInterface` crea un canal per llegir i escriure dades a la consola.
- `rl.question` mostra la pregunta i espera que l'usuari introdueixi una resposta.
- La resposta es passa a una funció de *callback*, on podem processar-la i emmagatzemar-la en una variable.

A més, podem capturar dades de forma més avançada, com per exemple llegir múltiples preguntes en seqüència o processar entrades contínues, però això ja forma part d'usos més avançats que estudiarem més endavant.

1.3.3 Operadors

JavaScript disposa d'**operadors** per a treballar amb tipus primitius i objectes. Ens permeten formar expressions. Els més habituals són els **aritmètics**, però també n'hi ha d'altres.

Operadors aritmètics

Serveixen per fer càlculs matemàtics:

```
1 5 + 2    // Suma → 7
2 5 - 2    // Resta → 3
3 (3 + 2) - 5 // Parèntesis → 0
4 3 / 3    // Divisió → 1
5 6 * 3    // Multiplicació → 18
```

L'operador + també serveix per **concatenar** cadenes de text:

```
1 "Hola " + "món" + "!" // Resultat: "Hola món!"
```

També tenim operadors d'**increment** i **decrement** (pre i post):

```
1 let x = 1;  
2 let y = ++x; // x = 2, y = 2  
3 let z = y++ + x; // z = 4, y = 3, x = 1
```

Operador typeof

Aquest operador especial ens permet saber **quin tipus de dada** conté una variable. És un operador *unari* (s'aplica a un sol operand):

```
1 typeof 5 // "number"  
2 typeof false // "boolean"  
3 typeof "Joan" // "string"  
4 typeof undefined // "undefined"
```

Operadors booleans

Només tenen dos valors possibles: `true` i `false`. Però poden modificar-se amb diversos operadors:

Negació

Canvia el valor a l'oposat:

```
1 !true // false  
2 !false // true  
3 !!true // true (doble negació)
```

Igualtat i identitat:

- `==` → Igualtat feble (comprova si són iguals, permet conversió de tipus)
- `===` → Identitat estricta (comprova valor i tipus)

```
1 true === true // true  
2 true === false // false  
3 true !== false // true  
4 true !== true // false
```

Es recomana fer servir `===` i `!==` en comptes de `==` i `!=` per evitar errors inesperats.

Error comú

Un error comú és pensar que aquesta expressió és falsa:

```
1 25 == "25"
```

En realitat són iguals, però no idèntics. L'expressió que retornaria `false` seria:

```
1 25 === "25"
```

Comparació

Podem comparar valors amb:

```
1 5 > 3      // true
2 5 < 3      // false
3 3 >= 3     // true
4 2 <= 1     // false
5 "a" < "b" // true
```

Valors que es consideren 'false' en JavaScript:

Valors falsy en JavaScript

En JavaScript, quan fem servir una expressió dins d'una condició (if, while, etc.), qualsevol valor es converteix implícitament a `true` o `false`. Els valors que s'avaluen com a `false` s'anomenen **falsy values**.

Els únics valors que es consideren **falsy** són:

- `false` (el valor booleà fals)
- `0` i `-0` (el zero numèric)
- `""` (cadena buida)
- `null`
- `undefined`
- `NaN` (Not a Number)
- `0n` (BigInt zero)

Tots els altres valors es consideren **truthy** (encara que siguin sorprenents, com per exemple `"0"`, `[]` o `{}.`

Operadors lògics

AND

Només retorna `true` si **tots dos valors** ho són:

```
1 true && true   // true
2 true && false  // false
```

Aquest operador (`&&`) s'utilitza molt per verificar condicions sense modificar els valors. Per exemple, és útil per comprovar si una propietat existeix.

La lògica que segueix és la següent:

- Si el **primer valor** és `false`, es retorna aquest valor.
- Si el **primer valor** és `true`, es retorna el segon valor.

```
1 0 && true // Retorna: 0 → perquè el 0 es considera un valor "
    fals"
2 1 && "Hola" // Retorna: "Hola" → perquè l'1 (o qualsevol valor
    diferent de 0) es considera "cert"
```

OR

Retorna el **primer valor cert**, o false si tots són falsos:

```
1 true || false // true
2 false || false // false
3 process.env.PORT || 5000 // Si process.env.PORT no està definit
    , s'utilitza 5000 com a valor per defecte
```

1.3.4 Classes Core i Mòduls de JavaScript

A més dels **tipus primitius** com number, string, boolean i undefined, JavaScript disposa de **classes bàsiques incorporades** que ens permeten treballar amb estructures de dades més complexes. Aquestes classes formen part del nucli del llenguatge (Core).

Les més habituals són:

- Object
- Number
- Date
- Math
- String
- Array

Totes elles **hereten de Object** i tenen els seus propis **mètodes i propietats**.

Object

És la classe més genèrica. Un objecte és un conjunt de **propietats (clau-valor)**, i pot contenir també **funcions** (anomenades mètodes).

```
1 let llibre = {
2   titol: "JavaScript per a tothom",
3   autor: "Ada Codaire",
4   pàgines: 240,
5   llegit: false
6 };
7
8 llibre.titol // "JavaScript per a tothom"
```

```
9  llibre["pàgines"]    // 240
10 llibre.llegit = true; // assignació
```

Number

L'objecte Number representa **valors numèrics**, tant enters com decimals, i ofereix mètodes útils per manipular-los.

Creació de números:

```
1  let enter = 42;
2  let decimal = 3.1415;
3  let hexadecimal = 0xFF;    // 255 en decimal
4  let exponencial = 5.4e2;    // 540
```

També es pot crear amb el constructor:

```
1  let n = new Number(7);    // No gaire recomanable, millor forma
    primitiva
```

Valors especials:

```
1  1 / 0          // Infinity
2  -1 / 0         // -Infinity
3  "a" / 2        // NaN (Not a Number)
```

Mètodes de l'objecte Number:

```
1  let n = 2.5674;
```

- `n.toFixed(2)` → "2.57" (arrodoneix amb 2 decimals)
- `n.toExponential(2)` → "2.57e+0" (forma científica)
- `(15).toString(2)` → "1111" (binari)
- `(255).toString(16)` → "ff" (hexadecimal)

Els valors numèrics en JavaScript tenen un rang aproximat de:

- `Number.MAX_VALUE` → valor màxim ($\sim 1.79 \times 10^{308}$)
- `Number.MIN_VALUE` → valor mínim ($\sim 5 \times 10^{-324}$)
- `Number.NaN`, `Number.POSITIVE_INFINITY`, `Number.NEGATIVE_INFINITY`

Date

L'objecte Date ens permet **treballar amb dates i hores**.

Crear una data:


```
1 let ara = new Date();           // Data i hora actual
2 let dataEspecifica = new Date(2025, 6, 17); // 17 de juliol de
   2025 (mesos compten des de 0!)
```

Obtenir components d'una data:

```
1 ara.getFullYear(); // 2025
2 ara.getMonth();    // 6 (juliol)
3 ara.getDate();      // 17
4 ara.getDay();       // 4 (dijous, sent 0 = diumenge)
5 ara.getHours();     // hora actual
6 ara.getMinutes();   // minuts actuals
```

Format com a cadena:

```
1 ara.toString();      // "Thu Jul 17 2025 10:32:00 GMT+0200"
2 ara.toLocaleDateString(); // "17/7/2025"
```

Math

És un **objecte estàtic** que proporciona funcions i constants matemàtiques. No cal crear una instància (`new Math()` no s'utilitza).

Constants útils:

```
1 Math.PI      // 3.141592...
2 Math.E       // 2.718281...
```

Funcions matemàtiques bàsiques:

```
1 Math.round(6.7); // 7 (arrodoniment)
2 Math.floor(6.7); // 6 (arrodoniment a la baixa)
3 Math.ceil(6.1);  // 7 (arrodoniment a l'alça)
4 Math.abs(-5);    // 5 (valor absolut)
5 Math.pow(2, 3);  // 8 (potència)
6 Math.sqrt(16);   // 4 (arrel quadrada)
```

Aleatorietat:

```
1 Math.random(); // nombre aleatori entre 0 i 1
2 Math.floor(Math.random() * 10); // enter aleatori entre 0 i 9
```

Trigonometria i logaritmes:

```
1 Math.sin(Math.PI / 2); // 1
2 Math.cos(0);           // 1
3 Math.tan(Math.PI / 4); // ~1
4 Math.log(10);           // logaritme natural
```

Array

Un Array (matriu) és una **col·lecció ordenada de valors**. Aquests valors poden ser nombres, cadenes, booleans, objectes, altres arrays, etc.

```
1 let buida = [];  
2 let numeros = [1, 2, 3];  
3 let textos = ["Hola", "adeu"];  
4 let mesclat = [1, "dos", true, [4, 5]];
```

També es pot crear amb el constructor:

```
1 let altra = new Array(1, 2, 3);
```

En lliçons posteriors es treballaran els arrays de forma més exhaustiva.

String

Una String és una seqüència de caràcters. Es defineix amb cometes simples, dobles o *backticks* (template strings).

Creació de strings:

```
1 let salutacio = "Hola món";  
2 let nom = 'Joan';  
3 let complet = `Hola, ${nom}!`; // "Hola, Joan!"
```

Accés als caràcters:

```
1 salutacio[0]; // "H"  
2 salutacio.charAt(1); // "o"  
3 salutacio.length; // 8
```

Mètodes útils:

```
1 // Transforma el text a majúscules  
2 "Hola".toUpperCase(); // "HOLA"  
3 // Transforma el text a minúscules  
4 "Hola".toLowerCase(); // "hola"  
5 // Cerca un text donat dins una cadena de text i retorna 'true'  
  si el troba  
6 "Hola món".includes("món"); // true  
7 // Indica la primera posició de l'string donat dins una cadena  
  de text  
8 "Hola món".indexOf("m"); // 5  
9 // Retorna el text comprès entre el rang donat  
10 "Hola món".substring(0, 4); // "Hola"  
11 // Retorna el text sense els espais en blanc  
12 " text ".trim(); // "text"
```

Convertir un string en array:

```
1 let paraules = "un dos tres".split(" "); // ["un", "dos", "tres"]
```

1.3.5 Condicionals

Les estructures condicionals ens permeten **executar diferents blocs de codi** en funció d'una condició determinada. D'aquesta manera podem **controlar el flux del programa** segons diferents casos o estats.

Assignació condicional (Operador ternari)

És una forma **compacta** de fer una assignació en funció d'una condició:

```
1 condició ? valor_si_cert : valor_si_fals
```

Exemples:

```
1 let numero = 10;  
2 let mesGranQueCinc = numero > 5 ? "sí": "no";  
3 console.log(mesGranQueCinc) // sí
```

Sentència if

S'utilitza per executar codi només si una condició és certa:

```
1 let nota = 8;  
2  
3 if (nota >= 5) {  
4   console.log("Aprovat");  
5 }
```

Sentència if / else

Permet executar un bloc si la condició és **certa**, i un altre bloc si és **falsa**:

```
1 let nota = 8;  
2  
3 if (nota >= 5) {  
4   console.log("Aprovat");  
5 } else {  
6   console.log("Suspès");  
7 }
```

Sentència if / else if / else

Permet encadenar **diverses condicions**:

```
1 let nota = 8;  
2  
3 if (nota < 5) {  
4   console.log("Suspès");
```

```
5 } else if (nota < 7) {  
6   console.log("Bé");  
7 } else if (nota < 9) {  
8   console.log("Notable");  
9 } else {  
10  console.log("Excel·lent");  
11 }
```

Es van comprovant les condicions en ordre fins que se'n compleixi una.

Sentència switch

És una alternativa **més llegible** a múltiples if / else if. Comprova un valor i executa el bloc corresponent al cas:

```
1 let color = "groc";  
2  
3 switch (color) {  
4   case "vermell":  
5     console.log("vermell");  
6     break;  
7   case "groc":  
8     console.log("groc");  
9     break;  
10  case "verd":  
11    console.log("verd");  
12    break;  
13  default:  
14    console.log("El color no és ni vermell, ni groc, ni verd");  
15 }
```

Cal utilitzar *break* per evitar que s'executin els casos següents.

1.3.6 Bucles (Iteracions)

Els bucles ens permeten **repetir un bloc de codi** mentre es compleixi una condició. Són molt útils per **recórrer arrays**, repetir càlculs o generar estructures dinàmiques.

Tots els bucles tenen habitualment tres parts:

1. **Inicialització:** defineix la variable de control.
2. **Condició de permanència:** s'avalua abans o després de cada iteració.
3. **Actualització:** modifica la variable de control per avançar en el procés.

while

El bucle while executa el codi mentre la condició sigui **certa**:

```
1 let i = 1;
2
3 while (i <= 5) {
4   console.log(i);
5   i++;
6 }
7 // Resultat: 1 2 3 4 5
```

do...while

Aquest bucle **garanteix que el bloc s'executa com a mínim una vegada**, ja que la condició es comprova **després**:

```
1 let i = 1;
2
3 do {
4   console.log(i);
5   i++;
6 } while (i <= 5);
7 // Resultat: 1 2 3 4 5
```

for

El bucle for és el més habitual. Reuneix les tres parts en una sola línia:

```
1 for (let i = 0; i < 5; i++) {
2   console.log(i);
3 }
4 // Resultat: 0 1 2 3 4
```

Bona pràctica: evitar càlculs innecessaris

Evita repetir `.length` en cada iteració:

```
1 let array = [1, 2, 3, 4];
2
3 // Forma poc òptima
4 for (let i = 0; i < array.length; i++) {
5   console.log(array[i]);
6 }
7
8 // Forma òptima
9 for (let i = 0, len = array.length; i < len; i++) {
10  console.log(array[i]);
11 }
```

forEach (només per arrays)

És un **mètode dels arrays** que executa una funció per a cada element:

```
1 let valors = [10, 20, 30];
2
3 valors.forEach(function(valor, index) {
4   console.log("A la posició " + index + " hi ha: " + valor);
5 });
```

for...in (per a objectes)

Aquest bucle recorre **les propietats d'un objecte**:

```
1 let cotxe = {  
2   marca: "Toyota",  
3   model: "Yaris",  
4   any: 2020  
5 };  
6  
7 for (var clau in cotxe) {  
8   console.log(clau + ": " + cotxe[clau]);  
9 }  
10 // marca: Toyota, model: Yaris, any: 2020
```

També existeix for...of, que recorre valors directament (millor per a arrays, cadenes i col·leccions iterables).

```
1 for (let valor of [10, 20, 30]) {  
2   console.log(valor); // 10 20 30  
3 }
```


2. Programació amb funcions

En la programació, les funcions són una de les eines més potents i fonamentals. Permeten organitzar el codi en blocs reutilitzables, facilitar la lectura i el manteniment dels programes, i reduir la repetició de codi. En JavaScript, les funcions són ciutadans de primera classe (first-class citizens), cosa que significa que poden ser tractades com qualsevol altra dada: es poden assignar a variables, passar com a paràmetres a altres funcions o fins i tot retornar-se com a resultat.

L'objectiu és entendre no només com es declaren i s'utilitzen les funcions, sinó també com pensar en termes de **modularitat i reutilització** —principis essencials en qualsevol llenguatge de programació modern.

2.1 Funcions predefinides del llenguatge

En tots els llenguatges de programació existeixen **llibreries de funcions** que serveixen per resoldre tasques habituals i repetitives. Aquestes llibreries estalvien als programadors haver d'escriure des de zero funcions comunes que sovint es necessiten en molts programes.

Un llenguatge de programació ben desenvolupat sol incloure **una bona quantitat de funcions i mètodes predefinits**, que faciliten molt la feina. De fet, en alguns casos pot ser més difícil **conèixer totes les llibreries** disponibles que aprendre la sintaxi bàsica del llenguatge.

En el cas de **JavaScript**, disposem de moltes funcions integrades. Com que és un llenguatge orientat a objectes, moltes d'aquestes funcions es troben **dins d'objectes** com Math (per operacions matemàtiques), String (per treballar amb cadenes), Array, Date, etc.

Tot i això, també hi ha **funcions globals** que no pertanyen a cap objecte concret i que podem utilitzar directament, com ara:

- `parseInt()`
- `parseFloat()`
- `isNaN()`
- `eval()`
- `alert()` o `console.log()`

Aquestes funcions **no necessiten conèixer cap objecte previ** i són especialment útils en les primeres etapes d'aprenentatge, ja que ens permeten **interactuar**

amb el llenguatge sense necessitat d'aprendre encara l'estructura completa dels objectes.

En aquesta lliçó ens centrarem justament en aquest tipus de funcions: les que **formen part del nucli de JavaScript i es poden utilitzar de forma directa.**

parseInt()

Converteix una cadena de text (String) en un **nombre enter**. Si la cadena comença amb caràcters no numèrics, retorna NaN.

```
1 parseInt("42");           // 42
2 parseInt("101", 2);       // 5 (base binària)
3 parseInt("3.14");         // 3
4 parseInt("abc");          // NaN
```

parseFloat()

Converteix una cadena de text en un **nombre decimal (coma flotant)**. També pot convertir nombres amb notació exponencial.

```
1 parseFloat("3.1416");     // 3.1416
2 parseFloat("10e2");        // 1000
3 parseFloat("5.5kg");       // 5.5
4 parseFloat("hola");        // NaN
```

isNaN()

Comprova si un valor **no és un número**. Retorna true si el valor és NaN, i false en cas contrari.

```
1 isNaN("hola");            // true (no és un número)
2 isNaN(5);                  // false
3 isNaN(NaN);                // true
4 isNaN("123");              // false (es pot convertir a número)
```

`isNaN()` fa una conversió prèvia a nombre. Si es vol evitar això, podem usar `Number.isNaN()` (més estricte).

eval()

Executa codi JavaScript que estigui contingut en una **cadena de text**. És molt potent, però perillós si no es controla bé (pot executar codi maliciós).

```
1 eval("2 + 2");             // 4
2 eval("var x = 5");         // defineix la variable x
3 console.log(x);            // 5
```

console.log()

Mostra informació a la **consola del navegador** o entorn d'execució. És molt útil per depurar (comprovar valors, traçar execució...).

```
1 let nom = "Joan";
2 console.log("Hola, " + nom); // Hola, Joan
```

alert()

Mostra una **finestra emergent (popup)** amb un missatge. S'utilitza per **avisos simples**, però no és recomanable en aplicacions modernes.

```
1 alert("Benvingut al curs de JavaScript!");
```

Quan s'executa, bloqueja l'execució del programa fins que l'usuari tanqui l'avís.

No volem donar una impressió equivocada amb aquesta petita llista de funcions natives de JavaScript. En realitat, existeixen moltes més funcions que anirem veient al llarg d'aquest material.

La diferència és que moltes d'aquestes funcions estan associades a objectes concrets. Per exemple:

- Les funcions per treballar amb **cadenes de caràcters** formen part de l'objecte `String`.
- Les funcions per fer **càlculs matemàtics avançats** estan dins de l'objecte `Math`.
- També hi ha funcions relacionades amb el **document HTML** (`document`) i amb la **finestra del navegador** (`window`).

A mesura que avancem en l'aprenentatge, **anirem descobrint aquestes funcions** i el seu ús pràctic dins del model d'objectes que ofereix JavaScript.

2.2 Funcions definides per l'usuari

Les **funcions en JavaScript** són blocs de codi executables als quals podem **passar paràmetres** i operar amb ells. Ens serveixen per **modularitzar els nostres programes** i estructurar-los en **blocs lògics que realitzin una tasca concreta**.

Això fa que el nostre codi sigui més **ordenat, llegible i fàcil de mantenir**, especialment quan el projecte creix.

2.2.1 Declaració de funció

Les funcions es declaren utilitzant la paraula reservada `function`, seguida habitualment d'un nom identificatiu per poder invocar-les més endavant en el codi.

Normalment, quan una funció finalitza la seva execució, retorna un valor mitjançant la paraula clau `return`. Aquest valor pot ser utilitzat per altres parts del programa.

```
1 function saludar(nom) {  
2   return "Hola, " + nom + "!";  
}
```

```
3 }  
4  
5 saludar("Joan"); // "Hola, Joan!"  
6 saludar("Marta"); // "Hola, Marta!"
```

Si no passem cap paràmetre:

```
1 saludar(); // "Hola, undefined!"
```

2.2.2 Funcions anònimes i assignació a variables

A JavaScript, podem crear **funcions sense nom** i assignar-les a una variable. Aquestes funcions s'anomenen **anònimes**, perquè no tenen identificador propi (no porten nom després de `function`).

Això ens permet **guardar la funció com si fos una dada més**, i després cridar-la utilitzant el nom de la variable.

```
1 // Assignem una funció anònima a la variable 'sumar'  
2 let sumar = function(a, b) {  
3   return a + b;  
4 };  
5  
6 // Ara podem cridar la funció com si fos una funció normal  
7 sumar(3, 4); // Retorna 7
```

2.2.3 Funcions fletxa (Arrow Functions)

Les **funcions fletxa** són una manera **més concisa** i moderna d'escriure funcions en JavaScript. Van ser introduïdes amb **ES6 (ECMAScript 2015)** i són molt utilitzades actualment, especialment en programació funcional i amb llibreries modernes com React o Vue.

Característiques principals:

- **Sintaxi més curta:** no cal escriure la paraula `function`.
- **Retorn implícit:** si el cos de la funció és una sola expressió, el resultat es retorna automàticament sense necessitat de `return`.
- **No tenen `this` propi:** hereten el `this` del context on han estat creades (important en funcions dins de mètodes o *callbacks*).
- No tenen arguments, `super` ni poden ser usades com a constructors (`new`).

Exemple bàsic:

```
1 const multiplicar = (a, b) => a * b;  
2  
3 multiplicar(3, 4); // 12
```

És equivalent a:

```
1 function multiplicar(a, b) {  
2   return a * b;  
3 }  
4  
5 multiplicar(3, 4); // 12
```

Quan només hi ha un paràmetre, podem **ometre els parèntesis**:

```
1 const doble = x => x * 2;
```

Equivalent a:

```
1 function doble(x) {  
2   return x * 2;  
3 }
```

Quan tenim zero, o més de dos paràmetres, cal mantenir els parèntesis:

```
1 const sumar = (a, b, c) => a + b + c;  
2 const mostrar = () => console.log("Hola món!");
```

Si la funció té més d'una línia o es vol fer servir return, cal obrir claus {}:

```
1 const mostrar = nom => {  
2   console.log("Hola " + nom);  
3   return true;  
4 };
```

Què entenem per "JavaScript modern"?

Quan parlem de **JavaScript modern**, ens referim a totes aquelles millores i noves funcionalitats que el llenguatge ha anat incorporant a partir de l'estàndard **ES6 (ECMAScript 2015)** i les versions posteriors. Aquestes novetats han fet que JavaScript sigui més potent, més fàcil d'escriure i més proper als llenguatges de programació actuals.

Alguns exemples d'aquest "JavaScript modern" són:

- **Funcions fletxa (arrow functions)** → sintaxi més curta i clara.
- **let i const** → millor control de l'abast (*scope*) de les variables.
- **Templates literals** → permeten inserir variables dins de cadenes amb `${}`.
- **Classes i mòduls** → per organitzar millor el codi i reutilitzar-lo.
- **Promeses i async/await** → nova manera d'abordar la programació asíncrona.

2.2.4 Paràmetres

Podem assignar valors per defecte en cas que no es passin arguments:

```
1 function saludar(nom = "amic") {  
2   return 'Hola, ${nom}';  
3 }  
4  
5 saludar(); // "Hola, amic"  
6 saludar("Pau"); // "Hola, Pau"
```

Encara que no especifiquem paràmetres, podem accedir als arguments passats amb arguments:

```
1 function llistar() {  
2   for (var i = 0; i < arguments.length; i++) {  
3     console.log(arguments[i]);  
4   }  
5 }  
6  
7 llistar("a", "b", "c");  
8 // Mostra: a, b, c
```

arguments només funciona
amb funcions normals, no
amb funcions fletxa.

2.2.5 Àmbit (Scope) de les variables

L'**àmbit (scope)** determina **on és visible una variable** dins del codi. Un bloc és qualsevol secció de codi delimitada per claus {}, com per exemple dins d'un if, for, while, function, etc. Les variables declarades dins d'una funció només són visibles en aquell entorn (àmbit local). Les de fora són globals.

Quan declarem una variable amb var, és **visible a tota la funció (àmbit global)** on es troba, encara que la definim dins d'un bloc. Això pot provocar errors difícils de detectar i és per això que no es recomana el seu ús.

Exemple:

```
1 function prova() {  
2   if (true) {  
3     var x = 5;  
4   }  
5   console.log(x); // 5 (tot i estar fora del bloc if)  
6 }
```

Les variables declarades amb let o const **només existeixen dins del bloc (àmbit local)** on han estat definides. Això evita errors per redefinicions accidentals o accessos indeguts.

Exemple amb let:

```
1 function prova() {
```

```
2   if (true) {  
3       let x = 5;  
4   }  
5   console.log(x); // Error: x is not defined  
6 }
```

Exemple amb const:

```
1 {  
2     const pi = 3.14;  
3 }  
4  
5 console.log(pi); // Error: pi is not defined
```

2.3 Conceptes avançats de funcions

En aquesta secció veurem com les funcions a JavaScript van molt més enllà de simples blocs de codi. Poden guardar informació del seu context (*closures*), actuar com a objectes de primera classe i fins i tot servir per crear altres funcions o objectes. També entendrem millor el paper del `this`, un concepte clau per treballar amb el context d'execució.

2.3.1 Closures (funcions tancades)

Un **closure** és una funció **que s'emporta amb ella l'entorn on va ser creada**, és a dir, **recorda les variables locals de la seva funció mare**, fins i tot després que aquesta hagi acabat d'executar-se.

Això permet:

- Encapsular dades privades que no es poden accedir directament des de fora.
- Crear funcions personalitzades amb valors fixats (com en configuradors, factories, etc.).

Anem a veure un exemple:

```
1 function crearComptador() {  
2     let comptador = 0;  
3  
4     return function() {  
5         comptador++;  
6         return comptador;  
7     };  
8 }  
9  
10 const cont1 = crearComptador();  
11 console.log(cont1()); // 1
```

```
12 console.log(cont1()); // 2
13
14 const cont2 = crearComptador();
15 console.log(cont2()); // 1
```

Cada `crearComptador()` té la seva pròpia còpia privada de comptador, que no és compartida amb altres instàncies. Fixem-nos que posem `cont1()` i no `cont1` perquè la funció `crearComptador()` retorna una funció.

2.3.2 Funcions com a objectes

A JavaScript, **les funcions són tractades com valors**, igual que els números, cadenes o arrays. Això vol dir que **una funció és un objecte de primera classe** (*first-class object*), cosa que ens permet:

- Assignar funcions a variables
- Passar funcions com a arguments a altres funcions
- Retornar funcions com a resultat
- Guardar funcions dins d'estructures de dades (arrays, objectes...)

Això obre la porta a una programació molt més flexible i potent, com per exemple:

- *Callbacks*
- *Closures*
- Funcions de més alt nivell
- Programació funcional

Què és la programació funcional?

La programació funcional és un estil de programació que tracta les **funcions com a primers ciutadanes** (*first-class citizens*). Això vol dir que les funcions no només serveixen per executar codi, sinó que també es poden **guardar en variables, passar com a arguments i retornar com a resultat** d'altres funcions.

Aquest enfocament permet escriure programes més **modulars, reutilitzables i fàcils de provar**, ja que cada funció és independent i no depèn de l'estat global.

Exemples típics en JavaScript:

- Passar una funció a `map()`, `filter()` o `forEach()`.
- Retornar funcions especialitzades mitjançant *closures*.
- Crear arrays d'operacions que podem executar dinàmicament.

Assignar funcions a variables:

```
1 const sumar = function(a, b) {
2   return a + b;
3 };
4
5 console.log(sumar(2, 5)); // 7
6 typeof(sumar); // function
```

Funció com a paràmetre:

```
1 function operar(op) {
2   return op(3, 4); // executem la funció que rebem com a parà
      metre
3 }
4
5 // Passem una funció fletxa com a argument
6 operar((a, b) => a * b); // 12
7
8 // També podem utilitzar una funció tradicional
9 operar(function(a, b) {
10   return a * b;
11 });
```

Aquí la funció `operar` rep una altra funció com a paràmetre (`op`) i la crida internament amb dos valors. Aquest patró és molt comú en *callbacks* i *frameworks* moderns.

Retornar funcions:

```
1 function crearOperadorMultiplicacio(x) {
2   return function(y) {
3     return x * y;
4   };
5 }
6
7 const perTres = crearOperadorMultiplicacio(3);
8 console.log(perTres(4)); // 12
```

En aquest cas, estem **retornant una funció que recorda el valor de x** (*closure*), creant una funció especialitzada en multiplicar per 3.

Guardar funcions dins d'un array:

```
1 const operacions = [
2   (a, b) => a + b,
3   (a, b) => a - b,
4   (a, b) => a * b,
5   (a, b) => a / b,
6 ];
7
8 console.log(operacions[2](6, 2)); // 12 (multiplicació)
9 // Accedim al tercer element de l'array
```

Les funcions també tenen propietats

Les funcions són objectes especials i poden tenir **propietats pròpies**:

Callback

Anomenem *callback* a una funció que passem com a paràmetre a una altra funció perquè s'executi més endavant, quan sigui necessari. S'utilitza molt en JavaScript per gestionar accions que triguen temps (com llegir dades, esperar un esdeveniment o treballar amb temporitzadors).


```
1 function saluda(nom) {  
2   return "Hola " + nom;  
3 }  
4  
5 saluda.descripcion = "Funció que saluda";  
6  
7 console.log(saluda.descripcion); // "Funció que saluda"
```

2.3.3 Funcions com a fàbriques (closures + mètodes)

Una **fàbrica de funcions** o **factory function** és una funció que crea i retorna un objecte amb mètodes propis. Aquesta tècnica permet **encapsular dades** mitjançant *closures* i retornar funcions que poden accedir-hi de manera segura.

Aquesta estratègia s'utilitza quan volem:

- Crear **objectes personalitzats** sense fer servir class.
- Mantenir **valors interns privats**.
- Exposar només una **interfície controlada** (funcions públiques).

Anem a veure un exemple on tindrem una variable **privada** (n) que només serà accessible dins de la funció comptador:

```
1 function comptador() {  
2   let n = 0; // variable privada  
3  
4   return {  
5     incrementar: () => ++n, // incrementa i retorna el nou  
6       valor  
7     decrementar: () => --n, // decrementa i retorna el nou  
8       valor  
9     valor: () => n // retorna el valor actual  
10  };  
11 }
```

Cada mètode és una **funció fletxa que forma part d'un closure**, i pot accedir a la variable `n`` gràcies al context tancat.

Ús del comptador:

```
1 const c = comptador();  
2  
3 c.incrementar(); // 1  
4 c.incrementar(); // 2  
5 c.valor(); // 2  
6  
7 c.decrementar(); // 1  
8  
9 // La variable n no és accessible des de fora  
10 console.log(c.n); // undefined
```

Això garanteix que **només es pot modificar n a través dels mètodes que hem dissenyat**, una manera molt neta d'encapsular dades.

Els avantatges d'aquest patró de disseny són:

- **Seguretat**: no exposem dades directament, només funcions controlades.
- **Reutilització**: podem crear múltiples instàncies independents.

```
1 const a = comptador();
2 const b = comptador();
3
4 a.incrementar(); // 1
5 b.incrementar(); // 1
6 a.incrementar(); // 2
7 b.valor();       // 1
```

Cada instància té **la seva pròpia còpia privada** de n.

2.3.4 El context "this"

En JavaScript, **this** és una **paraula clau** que fa referència al **context d'execució actual**, és a dir, **l'objecte que *posseix* la funció** quan s'executa.

Però **el valor de this canvia segons com s'invoca la funció**, i això pot ser confús si no ho coneixem bé.

Les funcions fletxa **no tenen el seu propi this**, sinó que prenen el **this** de l'àmbit on han estat creades. Això pot evitar errors en certs contextos com en *callbacks* dins d'objectes:

```
1 function Persona(nom) {
2   this.nom = nom;
3   setTimeout(() => {
4     console.log("Hola, soc " + this.nom); // correcte: this fa
5     // referència a Persona
6   }, 1000);
7 }
```

En canvi, amb la **function clàssica** no funcionaria igual:

```
1 function Persona(nom) {
2   this.nom = nom;
3   setTimeout(function() {
4     console.log("Hola, soc " + this.nom); // incorrecte: this no
5     // apunta a Persona
6   }, 1000);
7 }
```

En els exemples anteriors les funcions fletxa ens han anat bé perquè volíem accedir a una propietat que estava al mateix nivell de l'àmbit on havia estat creada.

Però a vegades ens pot jugar males passades. Fixem-nos en aquest exemple:

```
1 const obj = {  
2   nom: "Marc",  
3   normal: function () {  
4     console.log(this.nom); // "Marc"  
5   },  
6   fletxa: () => {  
7     console.log(this.nom); // undefined (this apunta a context  
8       global)  
9   },  
10 };  
11 obj.normal(); // "Marc"  
12 obj.fletxa(); // undefined
```

Quan cridem `obj.normal()`, la funció s'executa **com a mètode de l'objecte `obj`**, i per tant **`this` apunta a `obj`**. Això és el comportament normal que esperem.

Les **funcions fletxa no tenen `this` propi**. Quan les definim, **hereten el `this` del context en què van ser creades**.

La funció fletxa **no hereta el `this` de l'objecte `obj`**, perquè **l'objecte literal no crea un nou àmbit de `this`**. Això vol dir que `this` apunta **al context exterior**, que és el **context global** (o `undefined` en `use strict`).

Per tant, quan fem `obj.fletxa()`, el `this` no és `obj`, sinó el `this` global.

Quan convé fer servir arrow functions?

- Quan necessitem una funció curta i clara.
- Quan volem heretar el `this` del context actual.
- En *callbacks*, com `.map()`, `.filter()`, `.forEach()`, etc. (els veurem més endavant).

Quan no convé usar-les:

- Quan necessitem accedir a `this` d'un objecte creat amb `function` (ex. mètodes)
- Quan volem una funció constructora amb `new` (no és possible)